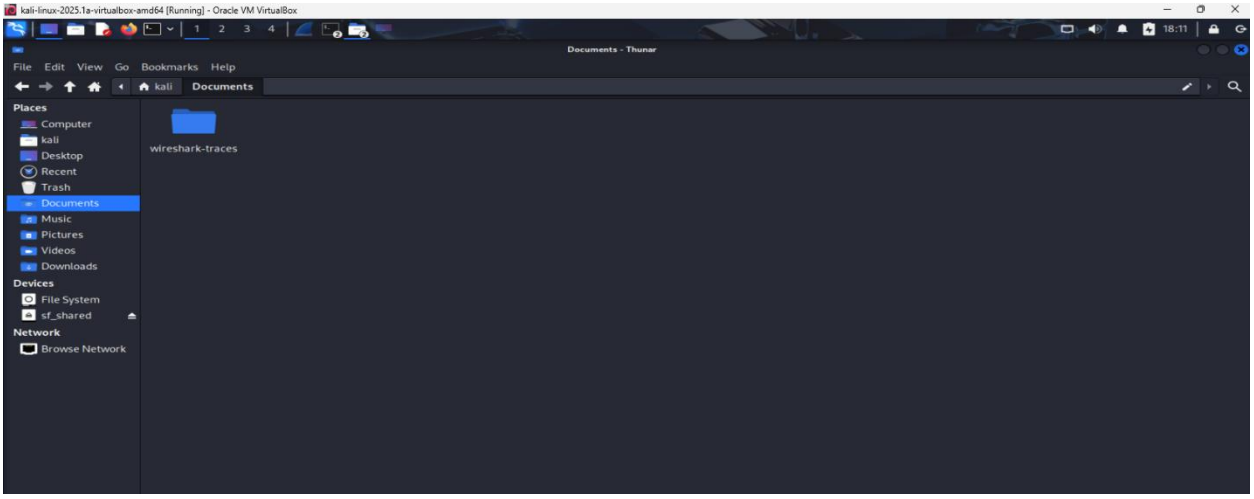
CSCE 5585 LAB 4

Advanced Wireshark Lab on NAT and Protocol Analysis

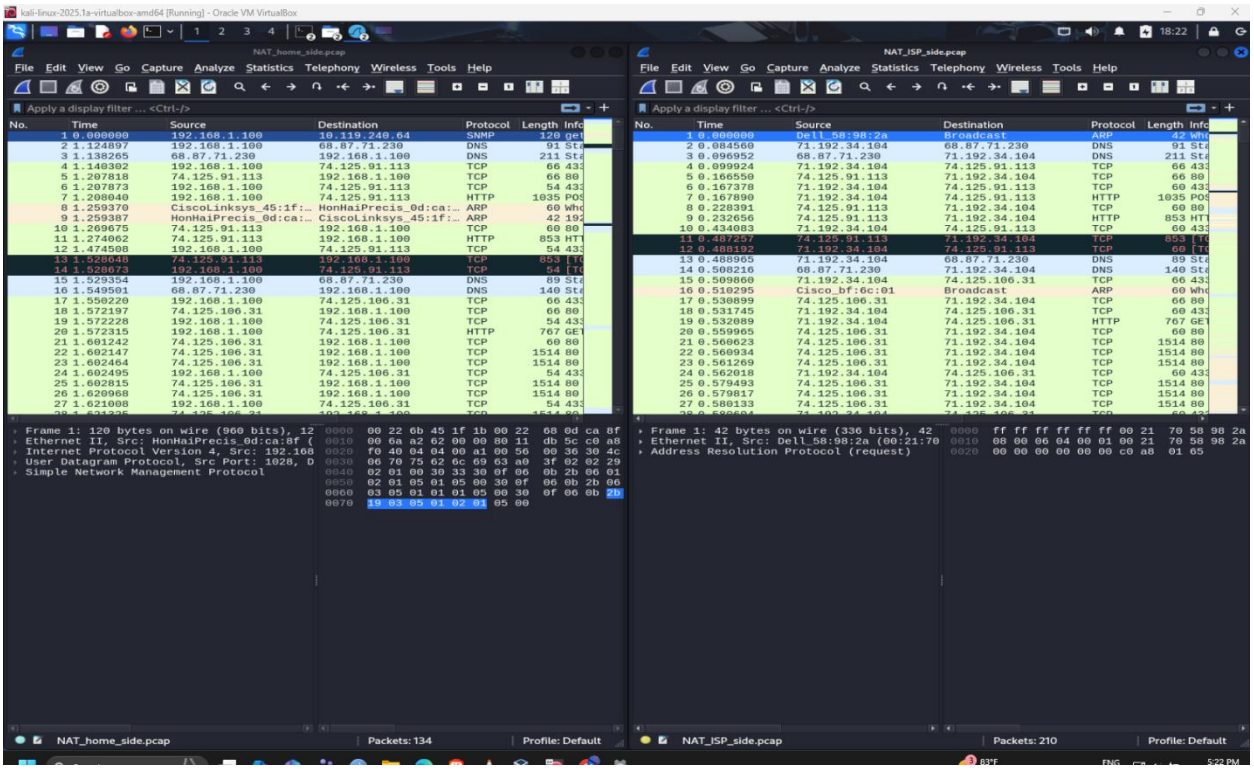
Chennamadhava Sri Vibhav Raju 11754947

Part1: NAT Measurement Scenarios

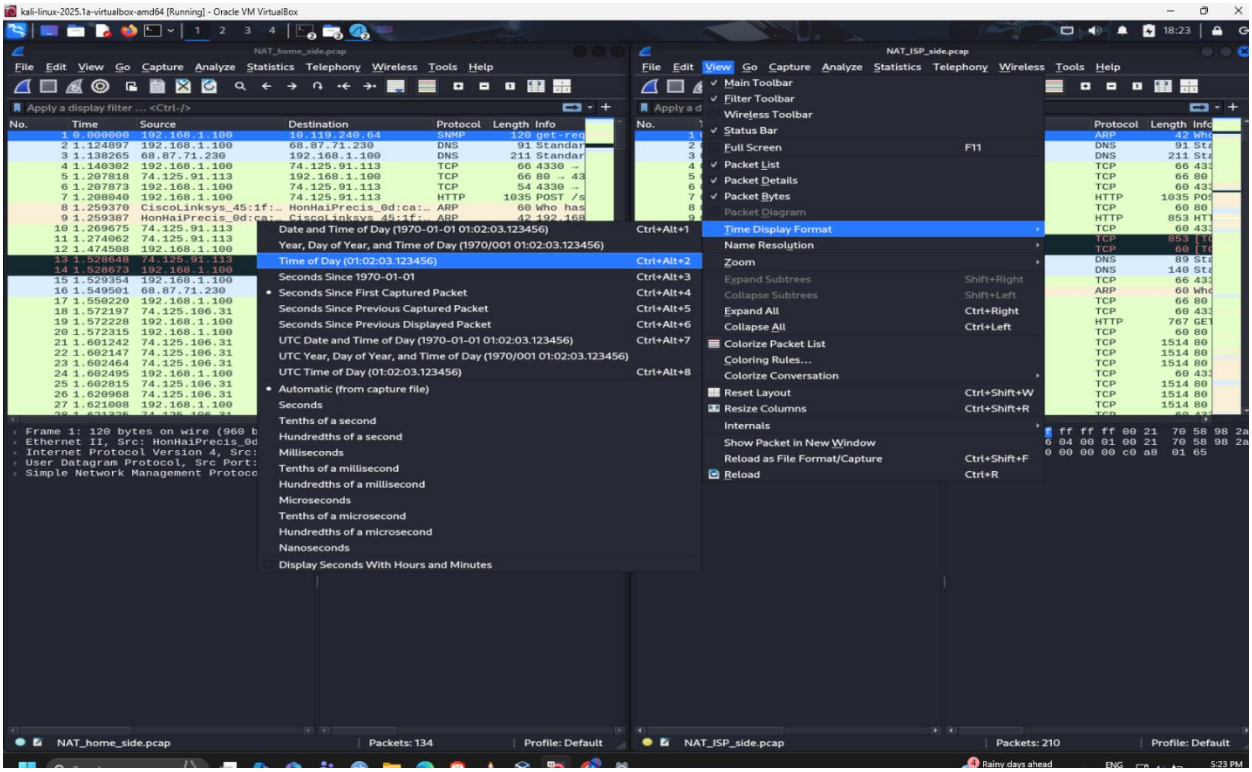
I have downloaded the wireshark logs from canvas which were provided



Opening logs both side by side NAT_home_side.pcap and NAT_ISP_side.pcap

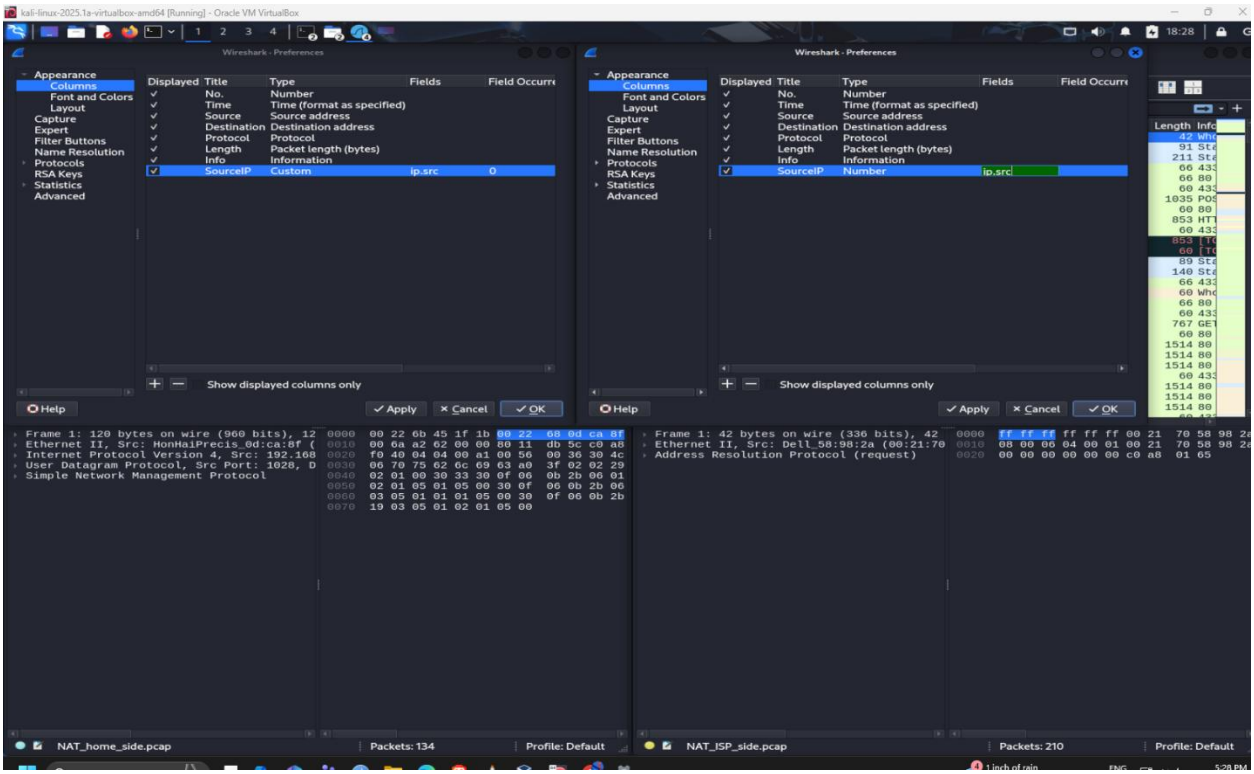


For correct format for comparing packet timing later.



Add required columns

Title: Source IP **custom column** ip.src



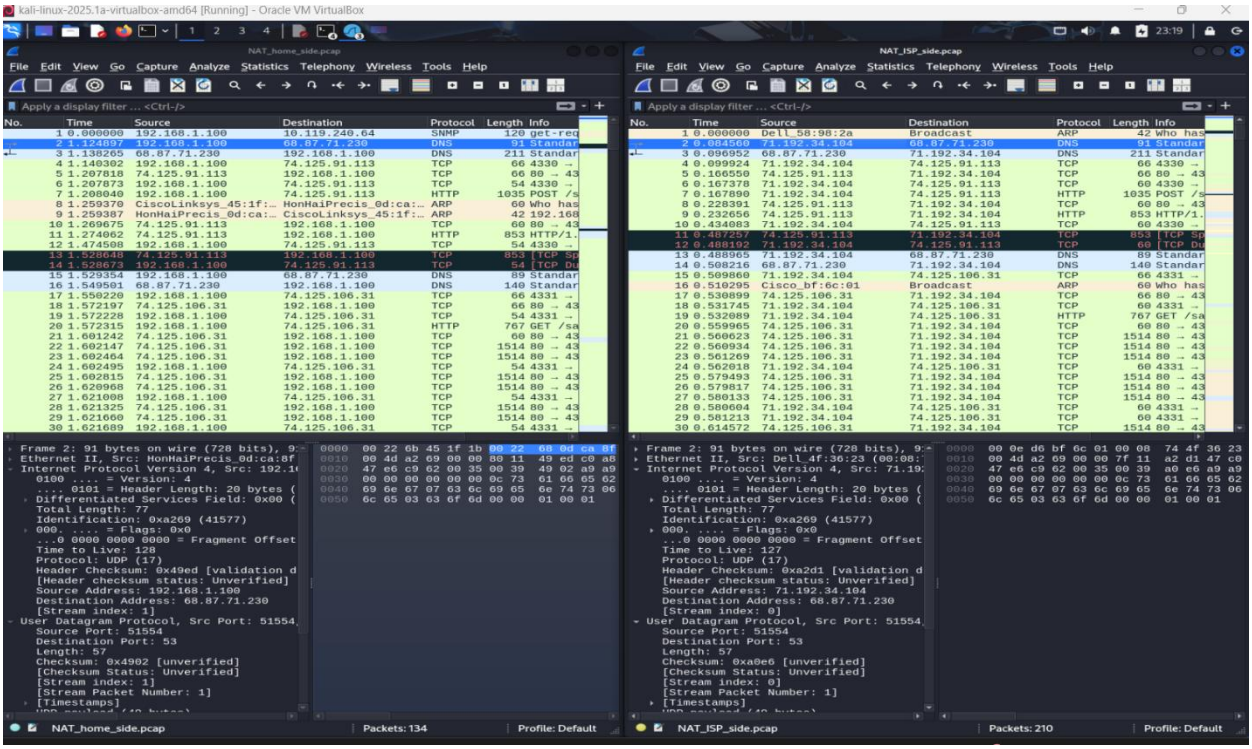
In comparing both the traces, the internal private IP of the client is displayed in the home-side capture (e.g. 192.168.1.100). The host on the ISP-side capture is the IP which is translated on the NAT (e.g. 71.192.34.104). This illustrates a classic many-to-one NAT mapping, in which all internal clienting traffic has been rewritten to a routable public address and sent over the ISP network. The destination server IP also does not change in both captures and proves that NAT only changes in the internal source identity but not endpoints.

Part 2: TCP Strem Analysis

IP Comparison and NAT Changes:

Analyzing the first SYN packet of the TCP stream of the captures, the only field to change is the source IP address that is not only rewritten but rewritten with the public address of the NAT, replacing the private address in the internal network. Other TCP header fields, such as the source port, destination port, sequence numbers, flags, window size and options are not lost. NAT maintains these fields to make sure that the end-to-end semantics of the TCP connection are not compromised, only the source IP should be rewritten in order to make the return traffic get the right path back to the internal host.

From the screenshot below



In comparing the initial SYN packet in the TCP stream of both captures, the field which has been altered by NAT is that of the source IP address. The SYN is sent by the home-side trace out of the private IP 192.168.1.100, but in the ISP-side trace the IP packet is received as a translated IP 71.192.34.104. The rest of the TCP header fields such as the source port (4335), destination port (80), sequence numbers, flags, window size as well as TCP options are all the same. NAT does not distort these fields to the end they maintain the end-to-end semantics of the TCP connection. The only alteration that needs to be done on the source IP is the need to ensure that the back traffic to the external server is rerouted properly to the internal host.

DNS Example (UDP Case)

Frame 2 of NAThomeside.pcap indicates:

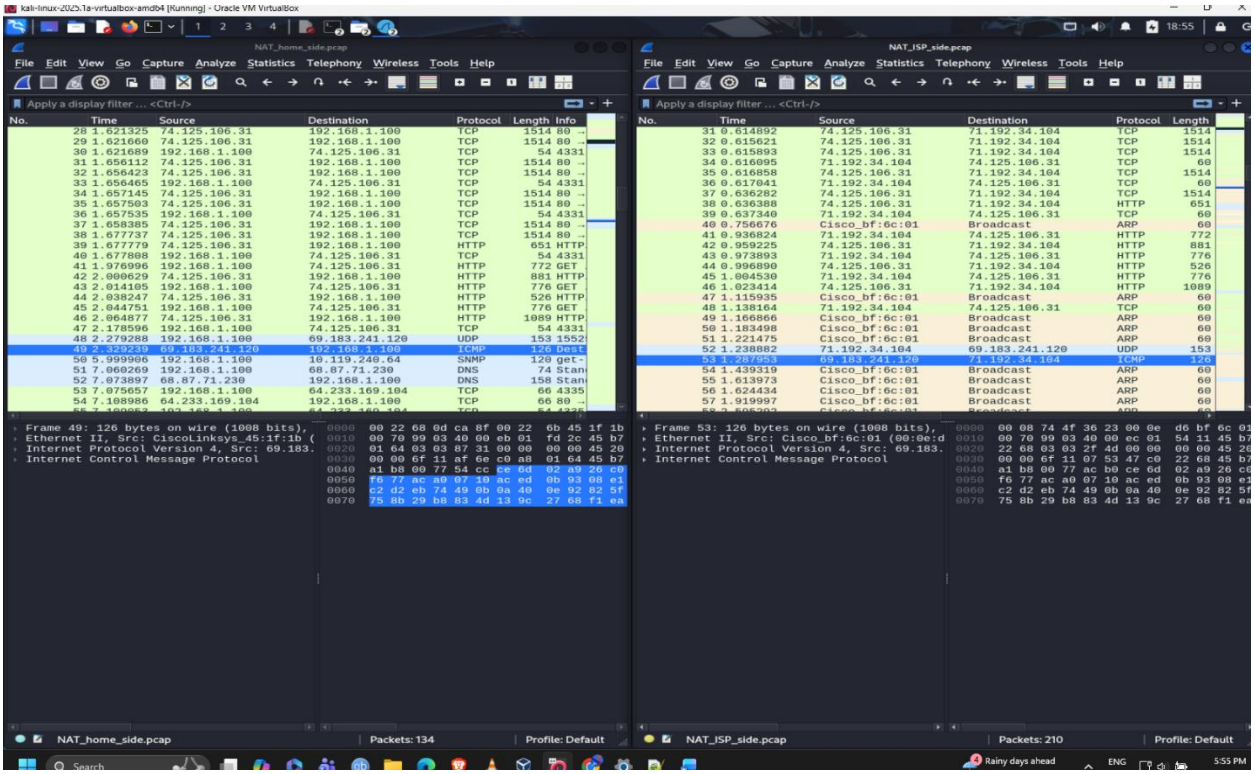
DNS 0xa9a9 Query: safebrowsing.clients.google.com

by 192.168.1.100 - 68.87.71.230.

The relevant packet can be found in NATISPside.pcap and it looks like:

68.87.71.230 - 71.192.34.104 sqrt -68.87.71.230 sqrt -68.87.71.230 sqrt -96.00.46.342 sqrt -96.00.46.342 sqrt -96.00.46.342 sqrt -96.00.46.342 sqrt -96.00.46

The destination IP and transaction ID match which shows that NAT did not change the UDP payload and header fields and only altered the source IP. This serves NAT correctly, i.e. rewrites addresses, not character at the application layer.



UDP + ICMP (Port Unreachable)

Home-side trace:

UDP packet: 192.168.1.100:15525 - 69.183.241.120:41400

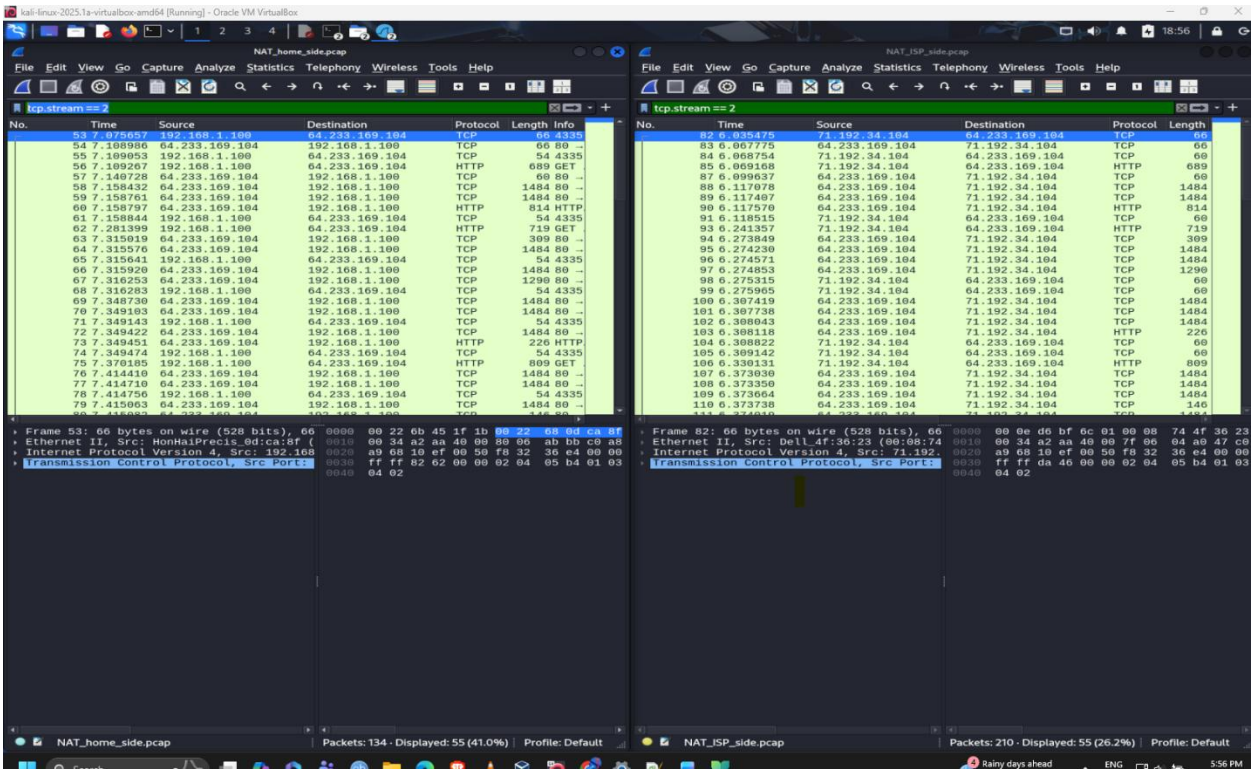
ICMP reply: 69.183.241.120 - 192.168.1.100 Destination Unreachable

ISP-side trace:

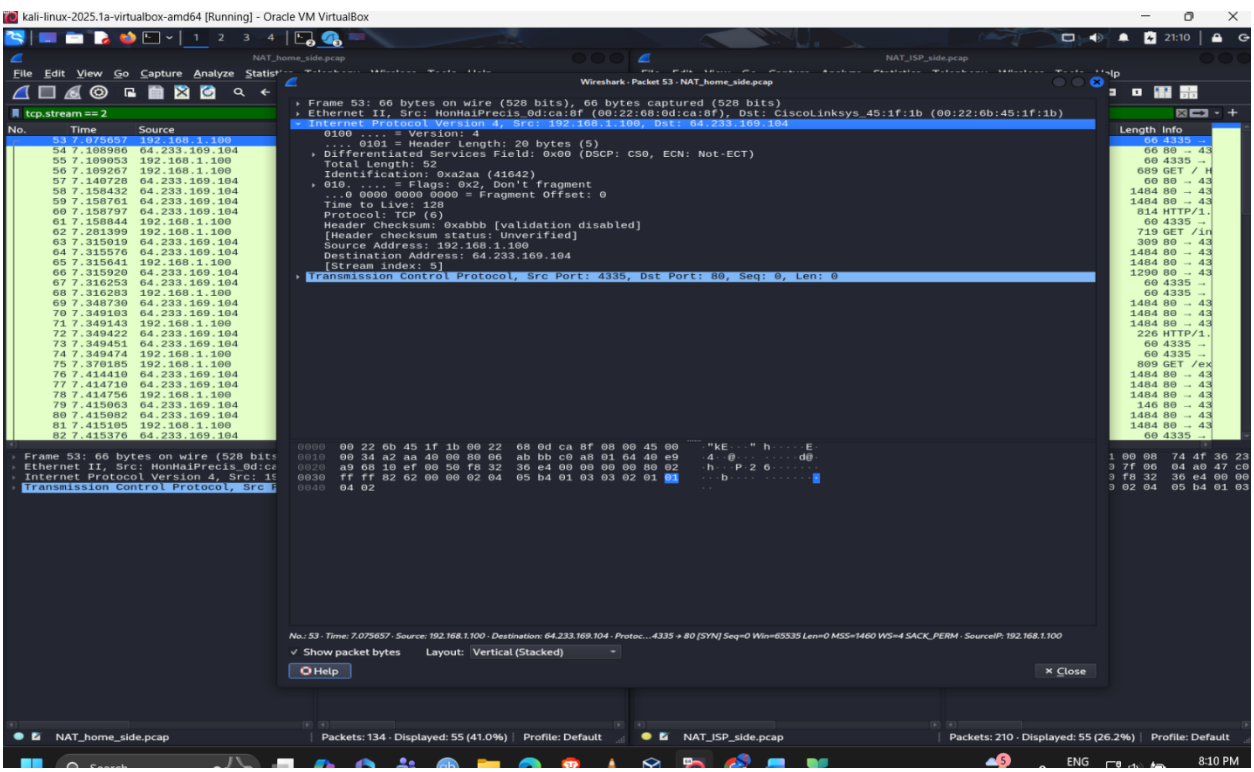
UDP packet translated to: 71.192.34.104:15525 - 69.183.241.120:41400

ICMP reply: 69.183.241.120 - 71.192.34.104

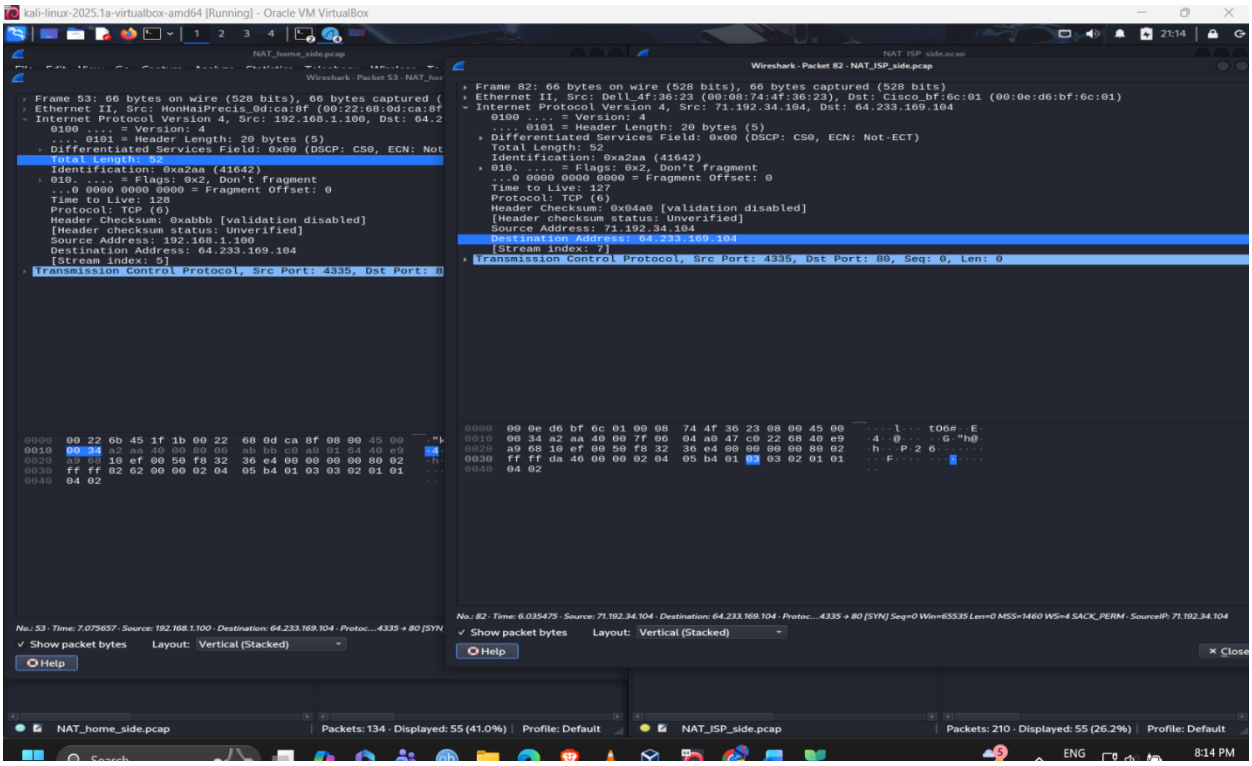
This demonstrates that even the error messages (ICMP) are initially returned to the public IP of NAT. The ICMP packet is then reverse translated and sent to the internal host by NAT. This represents that NATs uphold condition of UDP streams and safeguard proper routing of control-plane messages.



Homeside.pcap



Frame 53 7.075657 192.168.1.100 64.233.169.104 TCP 66 4335 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=4 SACK_PERM 192.168.1.100



ISP-side.pcap

Frame 82 6.035475 71.192.34.104 64.233.169.104 TCP 66 4335 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=4 SACK_PERM 71.192.34.104

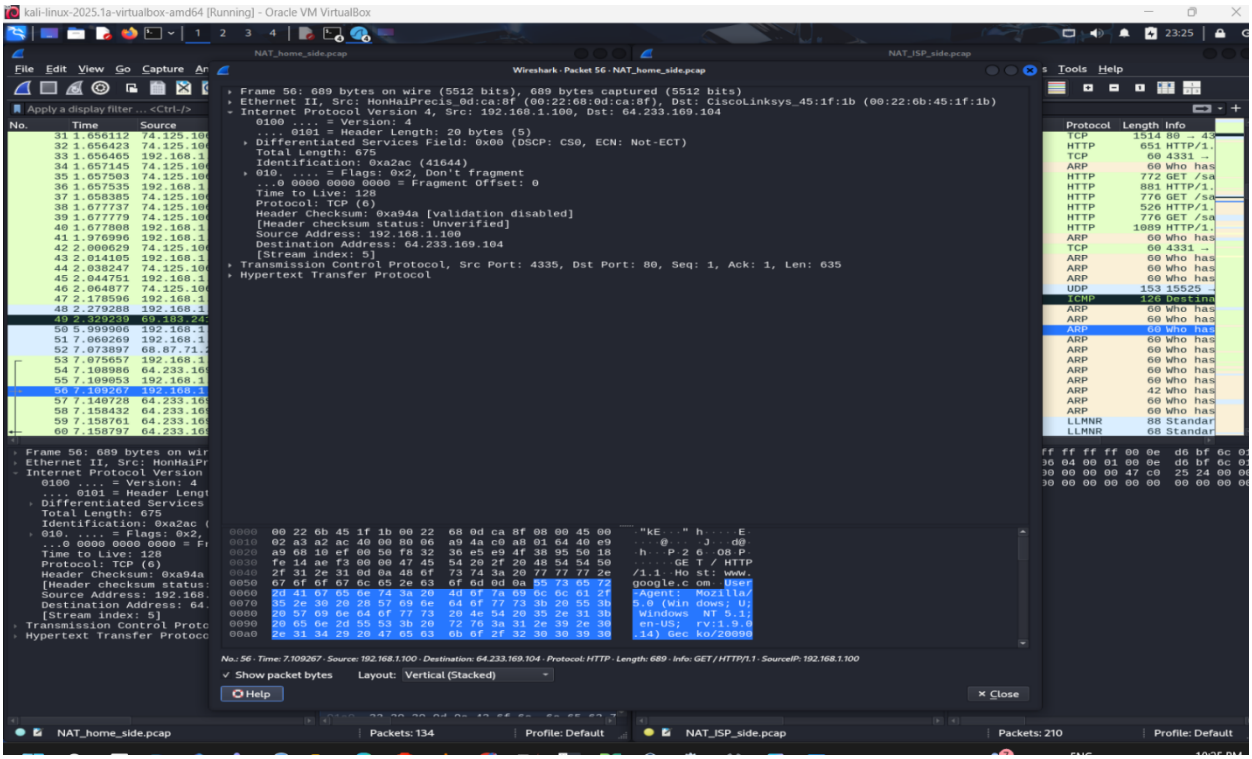
If we see and observe here we can conclude, after careful observation

NAT changed only the client IP address

192.168.1.100 → 71.192.34.104

The client port (4335) and the server IP/port stay the same.

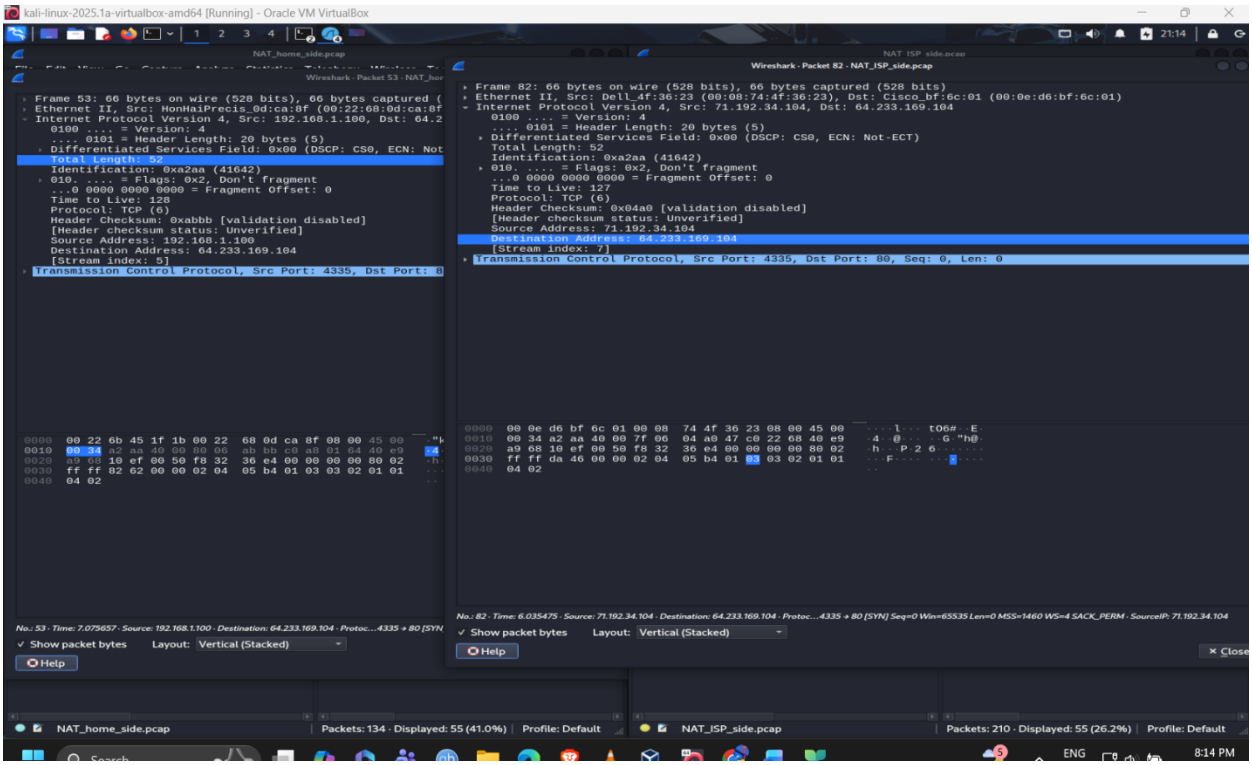
In other example HTTP GET /



Home: frame 56

56 7.109267 192.168.1.100 64.233.169.104 HTTP 689 GET / HTTP/1.1 192.168.1.100

192.168.1.100:4335 → 64.233.169.104:80 HTTP GET /



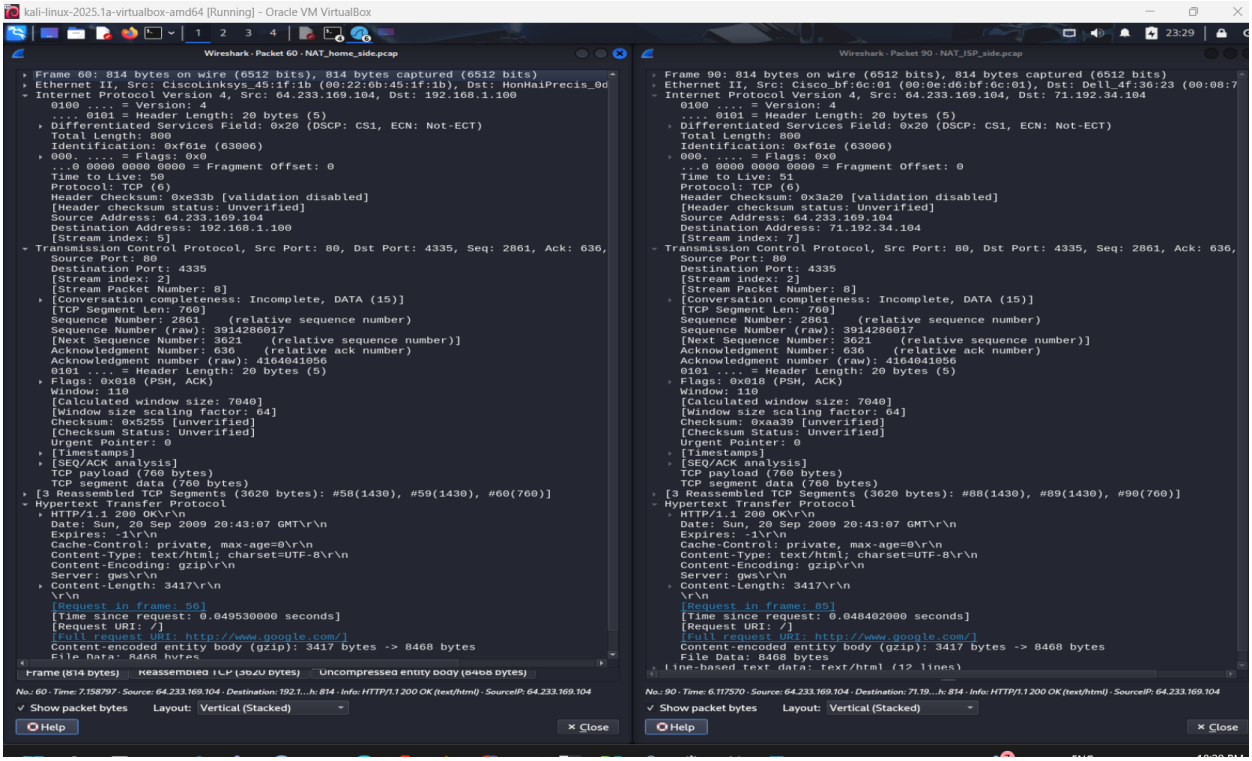
ISP: frame 85

85 6.069168 71.192.34.104 64.233.169.104 HTTP 689 GET / HTTP/1.1 71.192.34.104

71.192.34.104:4335 → 64.233.169.104:80 HTTP GET /

from other first HTTP 200 OK

Home: frame 60



60 7.158797 64.233.169.104 192.168.1.100 HTTP 814 HTTP/1.1 200 OK (text/html) 64.233.169.104

64.233.169.104 → 192.168.1.100 HTTP/1.1 200 OK

ISP: frame 90

90 6.117570 64.233.169.104 71.192.34.104 HTTP 814 HTTP/1.1 200 OK (text/html) 64.233.169.104

64.233.169.104 → 71.192.34.104 HTTP/1.1 200 OK

TCP SYN Example

Home-side:

192.168.1.100 - 64.233.169.104 TCP SYN port 4335 - 80

ISP-side:

71.192.34.104 - 64.233.169.104 TCP SYN port 4335 - 80

Again if we see that it is the source IP that is the only field that is NAT modified.

Examples of HTTP Request and Response.

HTTP GET / -- Home-side:

192.168.1.100:4335 - 64.233.169.104:80

ISP-side after translation:

71.192.34.104:4335 - 64.233.169.104:80

HTTP 200 OK -- Home-side:

64.233.169.104 - 192.168.1.100

ISP-side:

64.233.169.104 - 71.192.34.104

Summary of TCP Analysis.

Private/Internal: 192.168.1.100:4335

Translated/Public: 71.192.34.104:4335

Q. TCP Header analysis
the private/internal IP and port?
192.168.1.100:4335

the public/translated IP and port?
71.192.34.104:4335

Q The fields stay the same across the NAT?
Server IP 64.233.169.104, server port (80), client port (4335), TCP seq/ack numbers, payload (HTTP GET, HTTP 200), etc.
Field changes?
→ The client IP address (192.168.1.100 ↔ 71.192.34.104)

- **Private/Internal:** 192.168.1.100:4335
- **Translated/Public:** 71.192.34.104:4335
- **Unchanged fields:** ports, flags, window size, sequence and acknowledgment numbers, payload (HTTP), options.
- **Changed fields:** source IP address, TCP checksum (recomputed).

NAT must recompute the TCP checksum because the TCP checksum includes a **pseudo-header**, which incorporates the IP source address. Thus, when NAT changes the source IP, it must update the checksum accordingly.

IP Header Analysis

Still on the first SYN packet in each trace:

Internet Protocol Version 4.

Fields we are looking at

Source IP:

Home: 192.168.1.100

ISP: 71.192.34.104 changed by NAT

Destination IP: 64.233.169.104 (same)

Protocol: TCP (6) (same)

Total Length, ID: usually same or very close.

TTL: often decreased by 1 at the ISP side (router hop).

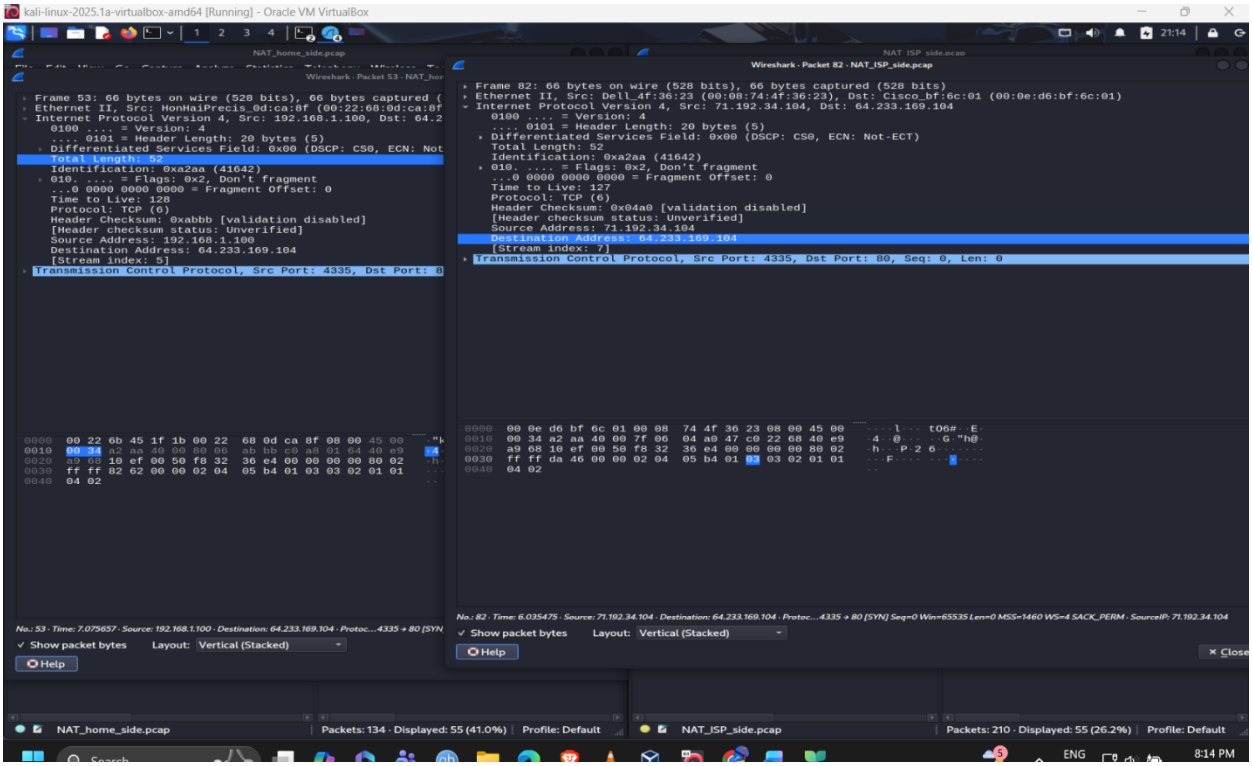
Header checksum: different (recomputed by router/NAT).

Source IP address, changed from private 192.168.1.100 to public 71.192.34.104 so the packet is routable on the Internet.

TTLdecreased by 1 because each router hop decrements TTL.

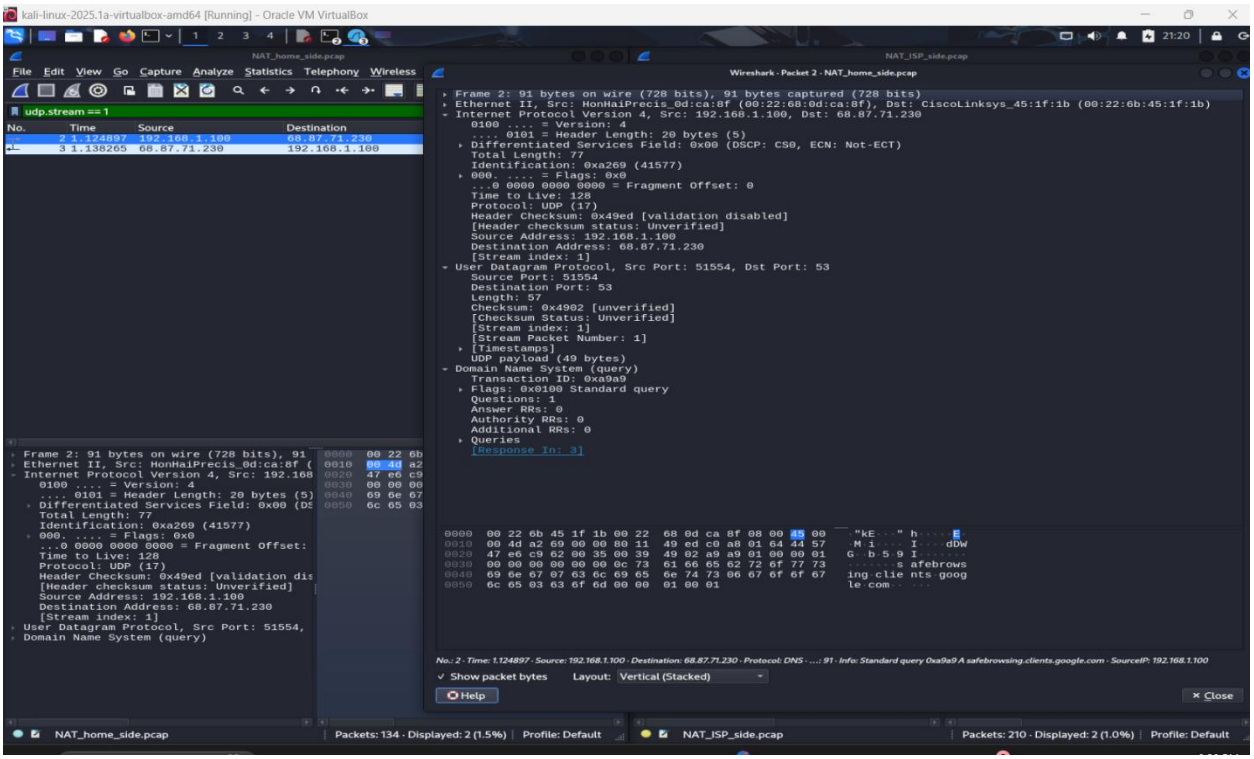
IP header checksum is recalculated because the header source IP and TTL changed, when NAT changes the source IP, it **must recompute** the checksum so the packet remains valid.

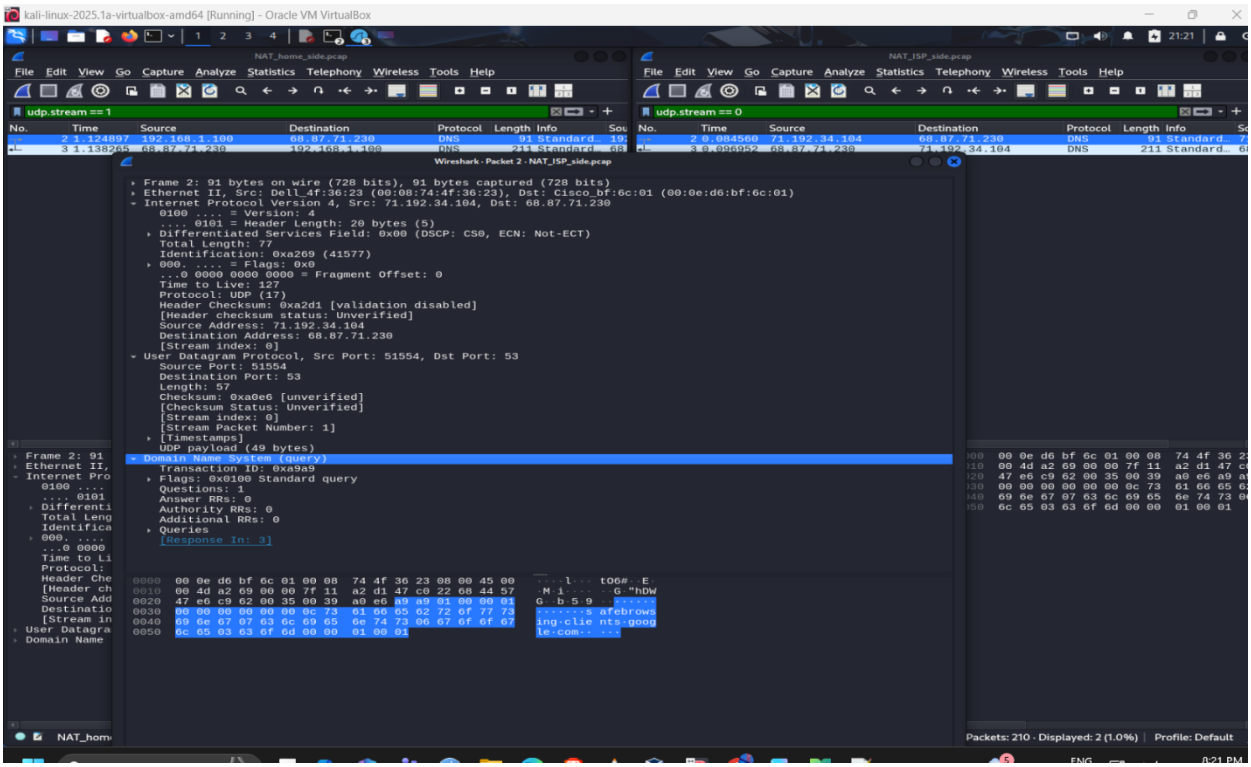
Everything else dest IP, protocol, total length is the same.



4. UDP Stream Analysis in NAT

Applying the filters udp.stream==1 to the home-side capture and udp.stream==0 to the ISP-side capture to isolate the UDP stream.





From above screenshots, home-side and ISP-side

Home-Side

- Source IP: 192.168.1.100
- Destination IP: 68.87.71.230
- Source Port: (DNS ephemeral) same as ISP side
- Destination Port: 53 (DNS)
- Length: same
- Flags: none (UDP has no flags)

ISP-Side

- Source IP: 71.192.34.104
- Destination IP: 68.87.71.230
- Source Port: **same as home-side** (important)
- Destination Port: 53
- Length: same
- The only field NAT changes in UDP is:

Checksum Analysis

Source IP (192.168.1.100 → 71.192.34.104)

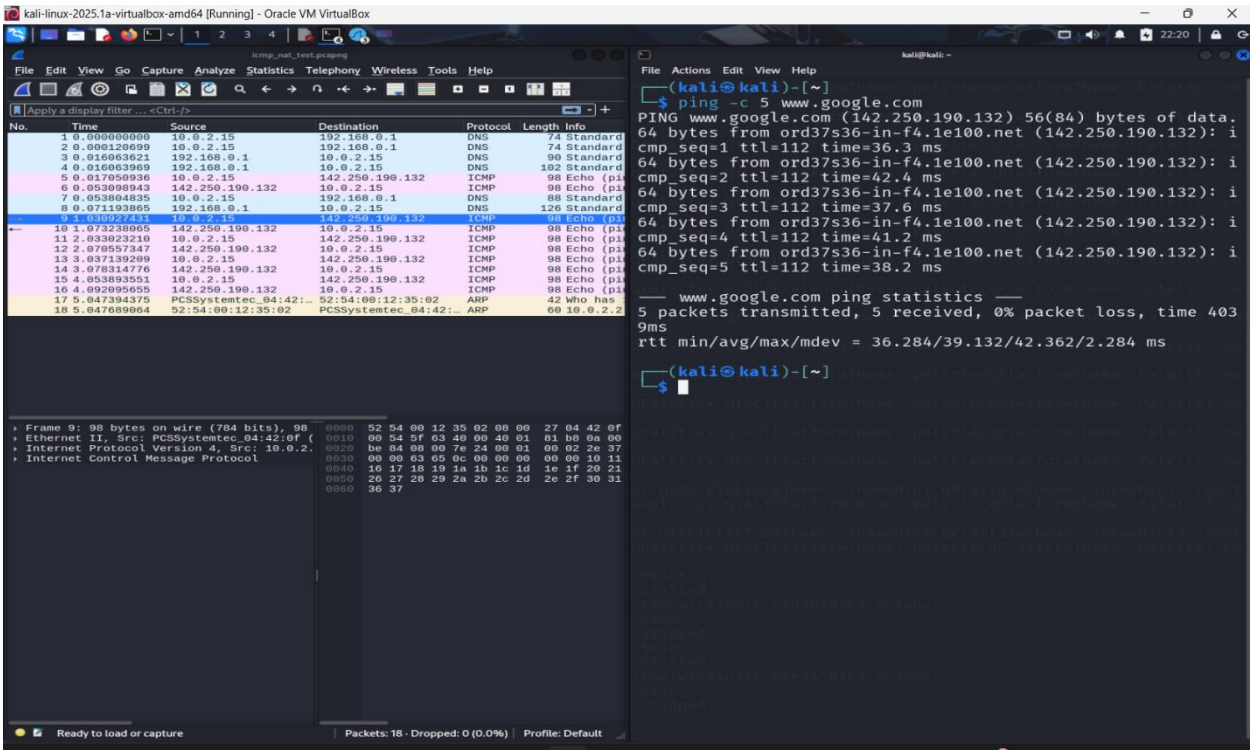
The source IP address is the only value that NAT alters in the UDP stream and is written differently as it is being changed mistakenly to the internal 192.168.1.100 with the external IP 71.192.34.104. The UDP header fields, including the source port, the destination port, the length, and the payload will all be the same in each of the captured packets on the home-side and on the ISP-side. The two captures however differ in the UDP checksum. This is because UDP makes use of pseudo-header in the computation of checksum which consists of source IP, destination IP, protocol value, and UDP length. In cases where NAT writes over the source IP, then one of the pseudo-header elements are altered necessitating a re-calculation of the checksum. Although the UDP data and port numbers will be the same, the checksum also needs to be changed to include the changed IP address in order to make the packet valid based on UDP integrity checks.

Additional tasks

NAT Timeouts & State Table – TCP Observation

- Wireshark capture interface eth0 on my vm
- I opened TCP connection and let it sit **idle for a long time** so I could see how long the NAT kept the state.

In the case of NAT Timeout analysis, I monitored network traffic using eth0 interface on my VM when connecting to another host using TCP connection and leaving it inactive over a long period. Through the packets I was able to see how long the NAT device held the connection state in state table. When the TCP connection was opened, NAT made an entry in the state listing the internal address and port as a private address and port as translated public address and port. This mapping has been maintained by the NAT even in long idle periods, indicating that TCP connections can take longer timeouts values due to being stateful and including explicit connection setup and teardown signals. The NAT continued to forward the return traffic properly as long as there was no case of going past a certain threshold limit. This experiment demonstrates the way NAT monitors active TCP flows and the impact of idle time on the timing of a mapping that may ultimately get expired.



Q1. ICMP Before vs. After NAT

NAT translation Before NAT translation, ICMP packets that are generated by the internal host have a private source IP address (e.g., 10.0.2.15). The same packet is reconstructed after going through the NAT device to utilize the NAT IP address that is exposed publicly at the ISP portion. Critically, NAT does not modify the identifiers used by ICMP including

identify numbers like Identifier or Sequence Number, as these are not included in the What separates12345 678910111213141516 here. Nevertheless, the ICMP checksum should be recalculated, since it contains a pseudo-header which is based upon some fields in the IP header, one of them being the source address. In this way, the payload is always preserved, but the NAT changes the address of the source at the IP-level and recontributes the checksum to be able to verify the packet integrity.

Q2. The difference between NAT and a TCP/UDP lying in the impact of ICMP.

NAT treats ICMP traffic differently as ICMP is not port-based unlike TCP and UDP which make use of source and destination ports to distinguish between flows. TCP/UDP tuples are stored with a 5-tuple, whereas ICMP traffic has no ports, instead NAT matches ICMP traffic using the ICMP Identifier field. ICMP sessions are by their very nature short lived, usually request response pairs, and as such NAT supports shorter state lifetimes than TCP. Also, ICMP is connectionless, stateless and NAT is more forgiving of the early loss of ICMP state information. This is in contrast to TCP where NAT must ensure that state is saved with great caution in order to ensure connections are reliable.

Q3. NAT State Tracking and Timeouts between TCP and UDP.

A NAT device has a state table which tracks all the flowing operations with a 5-tuple:

(IP source, IP source port, IP destination, IP destination port, protocol).

In case of TCP, NAT makes eagerness of the clear session limits formed by the SYN-SYN/ACK-ACK signal and FIN/ACK termination. Provided packets are flowing - even at intervals - NAT renews the TCP entry and it can survive several minutes or longer.

UDP is different in that it does not have a handshake, or even a teardown. NAT is unable to compute the init and end of the session and thus uses significantly lower values of the timeout (usually 30-60 seconds). When no UDP traffic updates the entry, then NAT discards the mapping. NAT has to generate a new state record on any subsequent UDP packets.

Q4. NAT Behavior When a Connection has to be restarted after the State Expiry.

When a connection gets interrupted by a NAT entry expiry, the next packet that cannot be associated with an adjacent mapping is considered as unknown traffic. With TCP, this normally causes the client to retransmit or create a new connection that creates a new three-way handshake and a new NAT mapping (which may have a different translated port).

In the case of UDP which is stateless, NAT merely initiates a new mapping as the subsequent outgoing packet comes along. UDP does not have any connection recovery facilities and therefore recovery is implicit and not explicit. In general, NAT will consider

traffic on an expired state as a new connection, and the connection should be successful.Q5. Filters to Reveal NAT Behavior

ip.src == 10.0.2.15

ip.src == 71.192.34.104

udp.port == 53

tcp.port == 80

kali-linux-2025.1a-virtualbox-amd64 [Running] - Oracle VM VirtualBox

*eth0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

ip.src == 10.0.2.15

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.2.15	192.168.0.1	DNS	70	Standa
3	0.013507315	10.0.2.15	192.168.0.1	DNS	70	Standa
7	25.755076590	10.0.2.15	142.250.190.78	TCP	74	48906
9	25.797857388	10.0.2.15	142.250.190.78	TCP	54	48906
10	188.362197188	10.0.2.15	142.250.190.78	TCP	54	48906
13	188.400883357	10.0.2.15	142.250.190.78	TCP	54	48906
16	227.428500974	10.0.2.15	1.1.1.1	DNS	47	Unknow
17	419.212366908	10.0.2.15	1.1.1.1	DNS	43	[Malfo
33	4018.3232406…	10.0.2.15	239.255.255.250	UDP/XML	844	56911
34	4018.3296215…	10.0.2.15	224.0.0.22	IGMPv3	54	Member
35	4018.4221889…	10.0.2.15	239.255.255.250	UDP/XML	844	56911
37	4018.6087222…	10.0.2.15	239.255.255.250	UDP/XML	844	56911
38	70694.622448…	10.0.2.15	239.255.255.250	UDP/XML	844	56911
43	70694.913578…	10.0.2.15	224.0.0.22	IGMPv3	54	Member

```
▸ Frame 17: 43 bytes on wire (344 bits), 43
▸ Ethernet II, Src: PCSSystemtec_04:42:0f (
▸ Internet Protocol Version 4, Src: 10.0.2.
▸ User Datagram Protocol, Src Port: 41493,
▾ [Malformed Packet: DNS]
  ▾ [Expert Info (Error/Malformed): Malforn
      [Malformed Packet (Exception occurre
      [Severity level: Error]
      [Group: Malformed]

0000  52 54 00 12 35 02 08 00  27 04 42 0f
0010  00 1d 51 b7 40 00 40 11  db 08 0a 00
0020  01 01 a2 15 00 35 00 09  0e 2b 0a
```

wireshark_eth0B95SF3.pcapn | Packets: 45 · Displayed: 14 (31.1%) · Dropped: 0 (0.0%) | Profile: Default

kali@kali: ~

File Actions Edit View Help

```
┌──(kali㉿kali)-[~]
└─$ ping -c 5 www.google.com
PING www.google.com (142.250.190.132) 56(84) bytes of data.
64 bytes from ord37s36-in-f4.1e100.net (142.250.190.132): i
cmp_seq=1 ttl=112 time=36.3 ms
64 bytes from ord37s36-in-f4.1e100.net (142.250.190.132): i
cmp_seq=2 ttl=112 time=42.4 ms
64 bytes from ord37s36-in-f4.1e100.net (142.250.190.132): i
cmp_seq=3 ttl=112 time=37.6 ms
64 bytes from ord37s36-in-f4.1e100.net (142.250.190.132): i
cmp_seq=4 ttl=112 time=41.2 ms
64 bytes from ord37s36-in-f4.1e100.net (142.250.190.132): i
cmp_seq=5 ttl=112 time=38.2 ms

--- www.google.com ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 403
9ms
rtt min/avg/max/mdev = 36.284/39.132/42.362/2.284 ms

┌──(kali㉿kali)-[~]
└─$ nc -lvp 4444
listening on [any] 4444 ...
hello
hi
^C

┌──(kali㉿kali)-[~]
└─$ nc -nv ggogle.com 80
Can't parse ggogle.com as an IP address

┌──(kali㉿kali)-[~]
└─$ nc -nv google.com 80
Can't parse google.com as an IP address

┌──(kali㉿kali)-[~]
└─$ nslookup google.com
Server:         192.168.0.1
Address:        192.168.0.1#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.191.142
Name:   google.com
```

kali-linux-2025.1a-virtualbox-amd64 [Running] - Oracle VM VirtualBox

*eth0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

udp.port == 53

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.2.15	192.168.0.1	DNS	70	Standa
2	0.010794181	192.168.0.1	10.0.2.15	DNS	86	Standa
3	0.013507315	10.0.2.15	192.168.0.1	DNS	70	Standa
4	0.028573817	192.168.0.1	10.0.2.15	DNS	98	Standa
16	227.428500974	10.0.2.15	1.1.1.1	DNS	47	Unknow
17	419.212366908	10.0.2.15	1.1.1.1	DNS	43	[Malfo

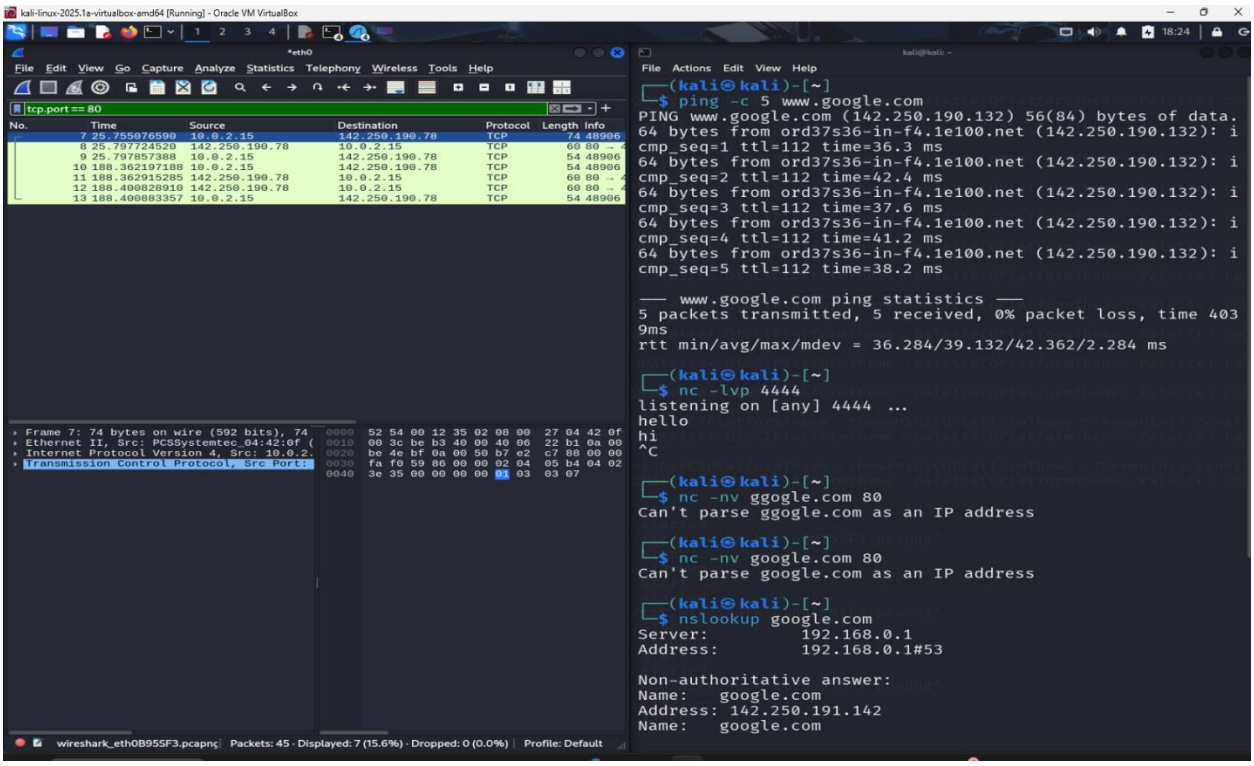
```
▸ Frame 1: 70 bytes on wire (560 bits), 70
▸ Ethernet II, Src: PCSSystemtec_04:42:0f (
▸ Internet Protocol Version 4, Src: 10.0.2.
▸ User Datagram Protocol, Src Port: 36329,
▸ Domain Name System (query)

0000  52 54 00 12 35 02 08 00  27 04 42 0f
0010  00 38 5c f1 00 00 40 11  51 0c 0a 00
0020  00 01 8d e9 00 35 00 24  cc ed 81 ee
0030  00 00 00 00 00 00 06 67  6f 6f 67 6c
0040  6d 00 00 01 00 01
```

wireshark_eth0B95SF3.pcapng | Packets: 45 · Displayed: 6 (13.3%) · Dropped: 0 (0.0%) | Profile: Default

kali@kali: ~

File Actions Edit View Help

```
┌──(kali㉿kali)-[~]
└─$ ping -c 5 www.google.com
PING www.google.com (142.250.190.132) 56(84) bytes of data.
64 bytes from ord37s36-in-f4.1e100.net (142.250.190.132): i
cmp_seq=1 ttl=112 time=36.3 ms
64 bytes from ord37s36-in-f4.1e100.net (142.250.190.132): i
cmp_seq=2 ttl=112 time=42.4 ms
64 bytes from ord37s36-in-f4.1e100.net (142.250.190.132): i
cmp_seq=3 ttl=112 time=37.6 ms
64 bytes from ord37s36-in-f4.1e100.net (142.250.190.132): i
cmp_seq=4 ttl=112 time=41.2 ms
64 bytes from ord37s36-in-f4.1e100.net (142.250.190.132): i
cmp_seq=5 ttl=112 time=38.2 ms

--- www.google.com ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 403
9ms
rtt min/avg/max/mdev = 36.284/39.132/42.362/2.284 ms

┌──(kali㉿kali)-[~]
└─$ nc -lvp 4444
listening on [any] 4444 ...
hello
hi
^C

┌──(kali㉿kali)-[~]
└─$ nc -nv ggogle.com 80
Can't parse ggogle.com as an IP address

┌──(kali㉿kali)-[~]
└─$ nc -nv google.com 80
Can't parse google.com as an IP address

┌──(kali㉿kali)-[~]
└─$ nslookup google.com
Server:         192.168.0.1
Address:        192.168.0.1#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.191.142
Name:   google.com
```



The applied filters we have applied show how packets look before NAT (private IP) and after NAT (public IP), have rewritten addresses and ports.

Q6. The assistance of these filtering methods to detect NAT-induced changes.

We may isolate traffic prior to and after NAT processing by applying specific Wireshark filters, including filtering by certain specific IPs in the private range (e.g. 192.168.0.1, 10.0.2.15), and filtered translated IPs in the public range (e.g 10.0.2.2, 10.0.2.6). When we compare the packet captured above the NAT boundary with the packet captured below the NAT boundary, it is easy to notice what fields are being rewritten.

Mostly, filters show contrasts on:

Private -public source IP address,

Port translation,Source port,

Checksum values (recalculated because of pseudo-header dependency),

TTL values (reduced by one hop).

These filters result in the easy possibility of comparisons of matched packet pairs to see how exactly the NAT device changes packets. This visibility of focus is vital in the knowledge of NAT behavior, confirmation of the accuracy of translation and ruling out routing or connections problems.

Q7. Compare private and public IP in applied NAT.

In a NAT environment under the real world scenario, a host within the local network would have a private IP address (e.g., 10.0.2.15 in a VM environment). As that packet leaves the router NAT substitutes the local source IP with the router-facing IP (e.g. 192.168.0.1 with local NAT or a global IP assigned by an ISP).

This is expected behavior, and the traces obtained prove it:

Home-side trace which has private IP + private ephemeral port.

beomes

Public IP + port on the ISP-side trace.

In the meantime, destination IP and port do not change allowing end-to-end transmissions.

The fact that the home-side and ISP-side agree on this address translation shows the NAT is properly implementing address translation and retaining all transport-layer semantics that are required to create successful routing and definitive connection integrity.